

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа

«Простые сортировки»

По дисциплине «Основы алгоритмизации и программирования»

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

Реализовать алгоритмы сортировки:

1. Сортировка слиянием.
2. Сортировка подсчетом.
3. Блочная сортировка.

АНАЛИЗ АЛГОРИТМА СОРТИРОВКИ СЛИЯНИЕМ

Для работы алгоритма следует реализовать следующие операции:

- Операцию для рекурсивного разделения массива на группы (метод sort);

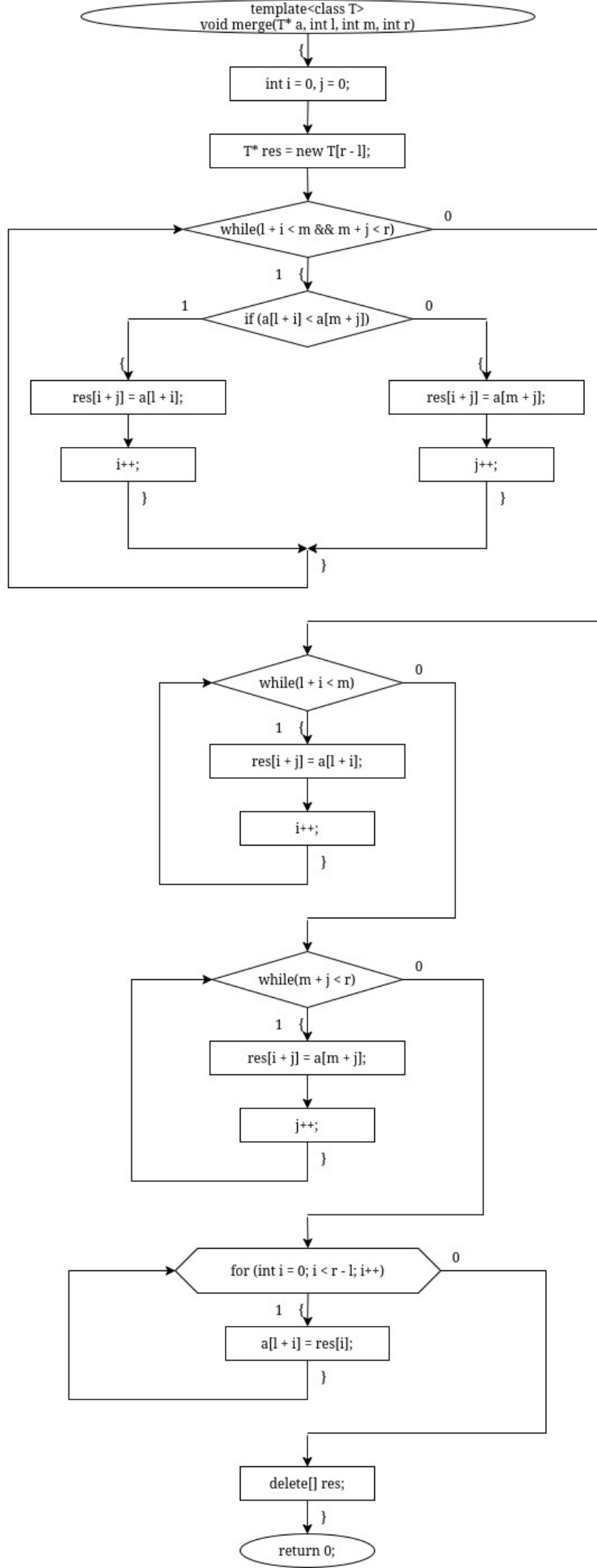
- Слияние в правильном порядке (метод merge);

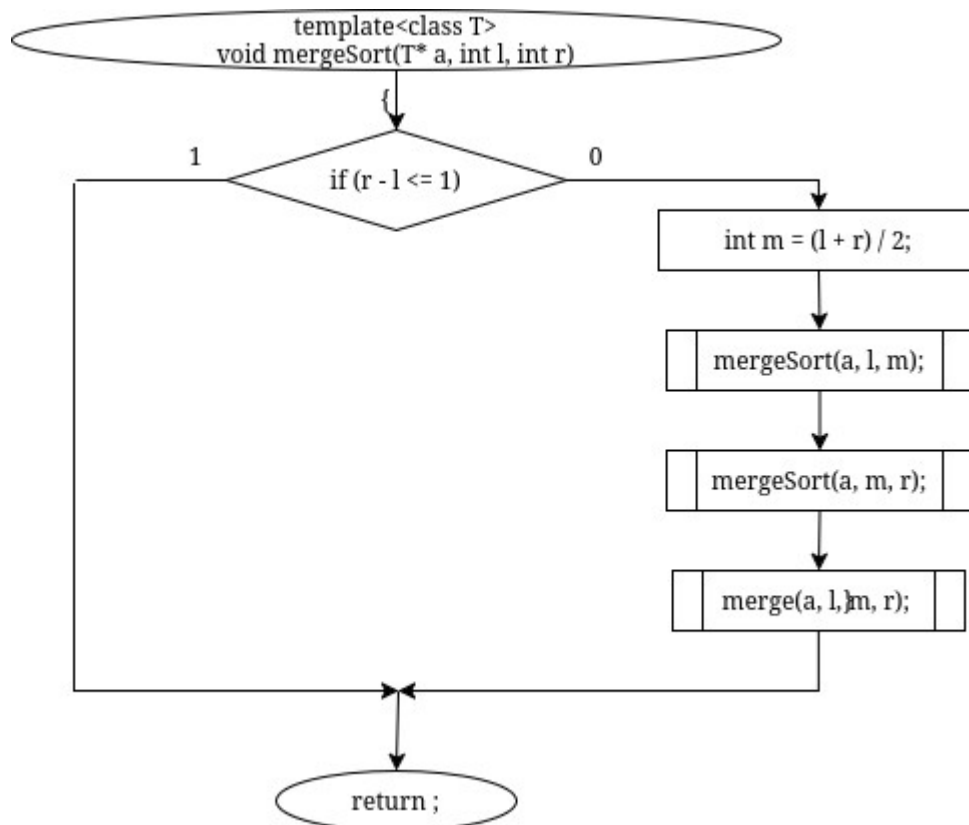
- Недостатки метода заключаются в том, что требуется дополнительная память по объему,

равной объему сортируемого файла. Поэтому для больших файлов проблематично

организовать сортировку слиянием в оперативной памяти.

БЛОК-СХЕМА

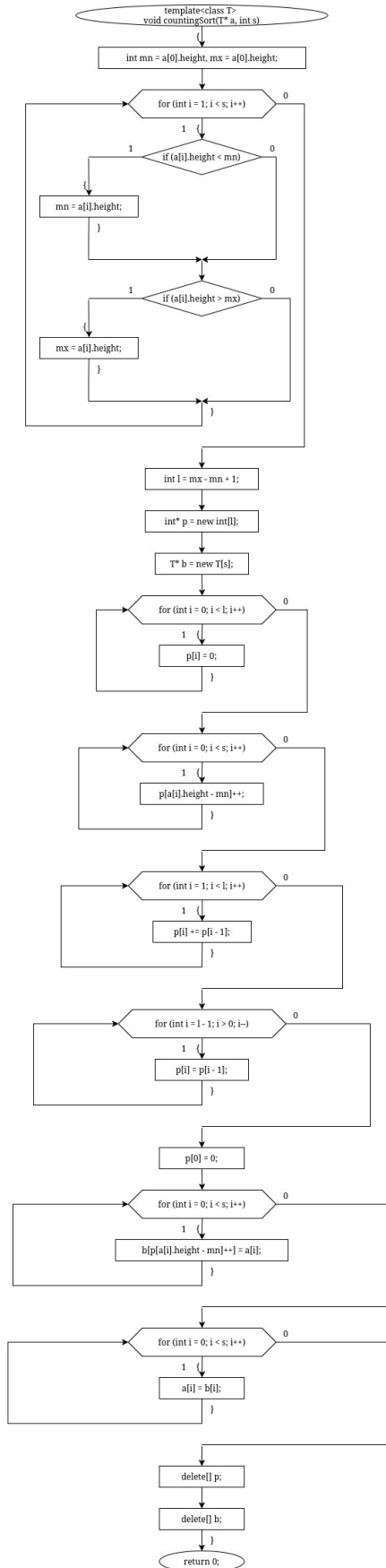




АНАЛИЗ АЛГОРИТМА СОРТИРОВКИ ПОДСЧЕТОМ

- Идея алгоритма состоит в предварительном подсчете количества элементов с различными ключами в исходном массиве и разделении результирующего массива на части соответствующей длины (блоки). Затем при повторном проходе исходного массива каждый его элемент копируется в специально отведенный его ключу блок, в первую свободную ячейку.
- Таким образом после завершения алгоритма в результирующем массиве содержится исходная последовательность в отсортированном виде, так как блоки расположены по возрастанию соответствующих ключей

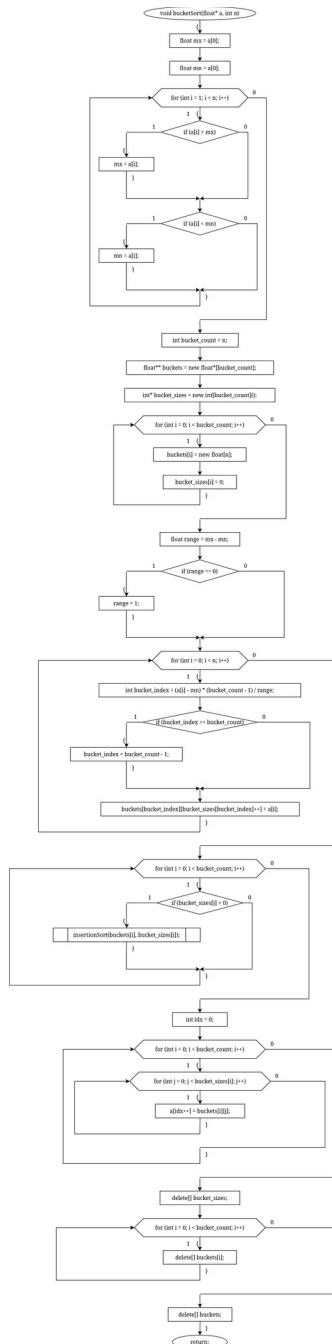
БЛОК-СХЕМА



АНАЛИЗ АЛГОРИТМА БЛОЧНОЙ СОРТИРОВКИ

Алгоритм блочной (корзинной) сортировки разделяет элементы массива входных данных на некоторое количество блоков - k , количество блоков зависит от количества исходного множества данных. Далее каждый из таких блоков сортируется либо другой сортировкой, либо рекурсивно тем же методом разбиения. После сортировок внутри каждого блока данные записываются в исходный массив в порядке разбиения на блоки.

БЛОК-СХЕМА



КОД main.cpp

```
#include <iostream>
using namespace std;

void insertionSort(float arr[], int n) {
    for (int i = 1; i < n; i++) {
        float key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

void bucketSort(float* a, int n) {
    float mx = a[0];
    float mn = a[0];
    for (int i = 1; i < n; i++) {
        if (a[i] > mx) mx = a[i];
        if (a[i] < mn) mn = a[i];
    }

    int bucket_count = n;
    float** buckets = new float*[bucket_count];
    int* bucket_sizes = new int[bucket_count]();

    for (int i = 0; i < bucket_count; i++) {
        buckets[i] = new float[n];
        bucket_sizes[i] = 0;
    }

    float range = mx - mn;
    if (range == 0) range = 1;

    for (int i = 0; i < n; i++) {
        int bucket_index = (a[i] - mn) * (bucket_count - 1) / range;
        if (bucket_index >= bucket_count) bucket_index = bucket_count - 1;

        buckets[bucket_index][bucket_sizes[bucket_index]++] = a[i];
    }

    for (int i = 0; i < bucket_count; i++)
```

```

    if (bucket_sizes[i] > 0)
        insertionSort(buckets[i], bucket_sizes[i]);

    int idx = 0;
    for (int i = 0; i < bucket_count; i++)
        for (int j = 0; j < bucket_sizes[i]; j++)
            a[idx++] = buckets[i][j];

    delete[] bucket_sizes;
    for (int i = 0; i < bucket_count; i++)
        delete[] buckets[i];
    delete[] buckets;
}

```

```

template<class T>
void merge(T* a, int l, int m, int r) {
    int i = 0, j = 0;
    T* res = new T[r - l];

    while (l + i < m && m + j < r) {
        if (a[l + i] < a[m + j]) {
            res[i + j] = a[l + i];
            i++;
        } else {
            res[i + j] = a[m + j];
            j++;
        }
    }

    while (l + i < m) {
        res[i + j] = a[l + i];
        i++;
    }

    while (m + j < r) {
        res[i + j] = a[m + j];
        j++;
    }

    for (int i = 0; i < r - l; i++)
        a[l + i] = res[i];
}

```



```
    delete[] res;  
}
```

```
template<class T>  
void mergeSort(T* a, int l, int r) {  
    if (r - l <= 1)  
        return;  
    int m = (l + r) / 2;  
    mergeSort(a, l, m);  
    mergeSort(a, m, r);  
    merge(a, l, m, r);  
}
```

```
template<class T>  
void countingSort(T* a, int s) {  
    int mn = a[0].height, mx = a[0].height;  
    for (int i = 1; i < s; i++) {  
        if (a[i].height < mn)  
            mn = a[i].height;  
        if (a[i].height > mx)  
            mx = a[i].height;  
    }  
}
```

```
int l = mx - mn + 1;
```

```
int* p = new int[l];
```

```
T* b = new T[s];
```

```
for (int i = 0; i < l; i++)
```

```
    p[i] = 0;
```

```
for (int i = 0; i < s; i++)
```

```
    p[a[i].height - mn]++;
```

```
for (int i = 1; i < l; i++)
```

```
    p[i] += p[i - 1];
```

```
for (int i = l - 1; i > 0; i--)
```

```
    p[i] = p[i - 1];
```

```
p[0] = 0;
```

```
for (int i = 0; i < s; i++)
```

```
    b[p[a[i].height - mn]++] = a[i];
```

```
for (int i = 0; i < s; i++)
```

```

        a[i] = b[i];

    delete[] p;
    delete[] b;
}

struct Person {
    std::string name;
    int height;

    void print() {
        cout << "\t\t" << name << ": " << height << "cm\n";
    }
};

int main() {
    cout << "Bucket sort\n";
    float bucket[] = {0.78, 0.17, 0.39, 0.26, 0.72, 0.94};
    int n = 6;

    cout << "\tДо сортировки:\n\t\t";
    for (int i = 0; i < n; i++) cout << bucket[i] << ' ';
    cout << endl;

    bucketSort(bucket, n);

    cout << "\tПосле сортировки:\n\t\t";
    for (int i = 0; i < n; i++) cout << bucket[i] << ' ';
    cout << endl;

    cout << "Counting sort\n";
    Person persons[] = {
        {"Алексей", 178},
        {"Мария", 165},
        {"Дмитрий", 182},
        {"Анна", 170},
        {"Иван", 165},
        {"Елена", 175}
    };

    cout << "\tДо сортировки:\n";
    for (auto &i: persons) i.print();

    countingSort(persons, 6);
    cout << "\n\tПосле сортировки:\n";

```

```

for (auto &i: persons) i.print();

cout << "Merge sort\n";
float doubles[] = {0.78, 0.17, 0.39, 0.26, 0.72, 0.94};

cout << "\tДо сортировки:\n\t\t";
for (int i = 0; i < n; i++) cout << doubles[i] << ' ';
cout << endl;

mergeSort(doubles, 0, n - 1);

cout << "\tПосле сортировки:\n\t\t";
for (int i = 0; i < n; i++) cout << doubles[i] << ' ';
cout << endl;

}

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

Bucket sort

До сортировки:

0.78 0.17 0.39 0.26 0.72 0.94

После сортировки:

0.17 0.26 0.39 0.72 0.78 0.94

Counting sort

До сортировки:

Алексей: 178см

Мария: 165см

Дмитрий: 182см

Анна: 170см

Иван: 165см

Елена: 175см

После сортировки:

Мария: 165см

Иван: 165см

Анна: 170см

Елена: 175см

Алексей: 178см

Дмитрий: 182см

Merge sort

До сортировки:

0.78 0.17 0.39 0.26 0.72 0.94

После сортировки:

0.17 0.26 0.39 0.72 0.78 0.94